Department of Computer Engineering - 2016-27

Artificial Intelligence Laboratory

[PCC304COM]

Term Work: 25 Marks

Practical : 25 Marks

Course Objectives:

1. To provide a comprehensive understanding of the fundamental concepts, principles, and methodologies of Artificial Intelligence.
2. To equip students with the ability to design and implement core AI algorithms for problem-solving, search, reasoning, and learning.
3. To develop analytical and practical skills for modeling and solving real-world problems using appropriate AI techniques.

Course Outcomes: Upon successful completion of this course, students will be able to:

- **CO1:** Explain and differentiate fundamental concepts, components, and techniques of Artificial Intelligence, including intelligent agents and AI methodologies.
- **CO2**: Design and implement AI algorithms for state-space search, heuristic search, reasoning, and basic learning tasks using programming tools.
- **CO3**: Model real-world problems as AI problems and apply appropriate AI techniques to obtain optimal or near-optimal solutions.
- **CO4**: To apply AI planning techniques for solving classical planning problems and relate them to real-world industrial AI applications.

Guidelines for Instructor's Manual

The instructor's manual is to be developed as a reference and hands-on resource. It should include prologue (about University/program/ institute/ department/foreword/ preface), curriculum of the course, conduction and Assessment guidelines, topics under consideration, concept, objectives, outcomes, set of typical applications/assignments/ guidelines, and references.

Guidelines for Student's Laboratory Journal

The laboratory assignments are to be submitted by student in the form of journal. Journal consists of Certificate, table of contents, and handwritten write-up of each assignment (Title, Date of Completion, Objectives, Problem Statement, Software and Hardware requirements, Assessment grade/marks and assessor's sign, Theory- Concept in brief, algorithm, flowchart, test cases, Test Data Set(if applicable), mathematical model (if applicable), conclusion/analysis). Program codes with sample output of all performed assignments are to be submitted as softcopy. As a conscious effort and little contribution towards Green IT and environment awareness, attaching printed papers as part of write-ups and program listing to journal must be avoided. Use of DVD containing students programs maintained by Laboratory In-charge is highly encouraged. For reference one or two journals may be maintained with program prints in the Laboratory.

Sr. No.	Practical
1	Design and implement a simple reflex agent for the Vacuum Cleaner world and analyze its rational behavior in a given environment.
2	Implement a model-based intelligent agent capable of navigating a grid environment with obstacles using internal state representation.
3	Implement A* Search for the 8 Puzzle Problem.
4	Implement Adversarial Search with Alpha-Beta Pruning.
5	Implement Tower of Hanoi by State Space Search.
6	Implement a Sudoku solver using backtracking and constraint propagation techniques.
7	Implement Local Beam Search for the Traveling Salesman Problem (TSP).

Sr. No.	Practical
1	Use BFS to solve a planning problem (e.g., 8 Puzzle Problem, Robot Path Planning, Blocks World).
2	Use DFS for problem solving (e.g., Water Jug Problem, Missionaries and Cannibals, Maze Solving, Blocks World).
3	Implement backtracking search to solve the 8-Queens Constraint Satisfaction Problem.
4	Develop a simple chatbot using predefined rules and pattern matching.
5	Design an intelligent agent that guesses a number using feedback (higher/lower).
6	Implement Goal Stack Planning for solving the Blocks World Problem and achieve a specified goal configuration.
7	Develop a game-playing agent using the Minimax algorithm for optimal decision-making.

Sr. No.	Mini Project
1	Medical Diagnosis System (Rule-Based Expert System) using forward and backward chaining.
2	Develop a simple rule-based recommendation system (e.g., movie or book suggestion system).

Practical NO-1

Aim :

Design and implement a simple reflex agent for the Vacuum Cleaner world and analyze its rational behavior in a given environment.

Objectives

1. To understand the concept of an Intelligent Agent.
2. To study the working of a Simple Reflex Agent.
3. To implement the Vacuum Cleaner World problem using Python.
4. To analyze the rational behavior of the agent in different environments.
5. To understand how an agent makes decisions based only on the current percept.

. Theory

- **Artificial Intelligence**
  Artificial Intelligence (AI) is a branch of computer science that enables machines to perform tasks that normally require human intelligence. These tasks include learning, reasoning, problem-solving, decision-making, and understanding the environment.
  One of the main goals of AI is to design **Intelligent Agents** that can perceive their environment and take appropriate actions to achieve specific goals.

- **Intelligent Agent**

An Intelligent Agent is an autonomous system that observes its environment through sensors and acts upon the environment using actuators to achieve a desired goal.

The general structure of an intelligent agent is:

Environment → Sensors → Agent → Actuators → Environment

Examples:

- Robot
- Self-driving car
- Chatbot
- Smart vacuum cleaner
- Voice assistant

➢ **Simple Reflex Agent**

A **Simple Reflex Agent** makes decisions based only on the **current state** using **IF–THEN rules**. It does not remember past actions, so it is called a **Memoryless Agent**.

Example:

- IF room is Dirty → **SUCK**
- ELSE → **Move**
- **3.4 Vacuum Cleaner World**
- The **Vacuum Cleaner World** is a simple AI model with **Room A**, **Room B**, and a **Vacuum Cleaner Agent**. Each room can be **Clean** or **Dirty**. The goal is to clean all dirty rooms with minimum actions.

➢ **Components**

- **Environment:** Room A and Room B
- **Agent:** Vacuum Cleaner
- **Sensors:** Current room and its status
- **Actuators:** SUCK, MOVE LEFT, MOVE RIGHT

➢ **Working of the Agent**

1. If the current room is dirty, **clean it**.
2. If the room is clean, **move to the next room**.
3. Repeat until **both rooms are clean**.

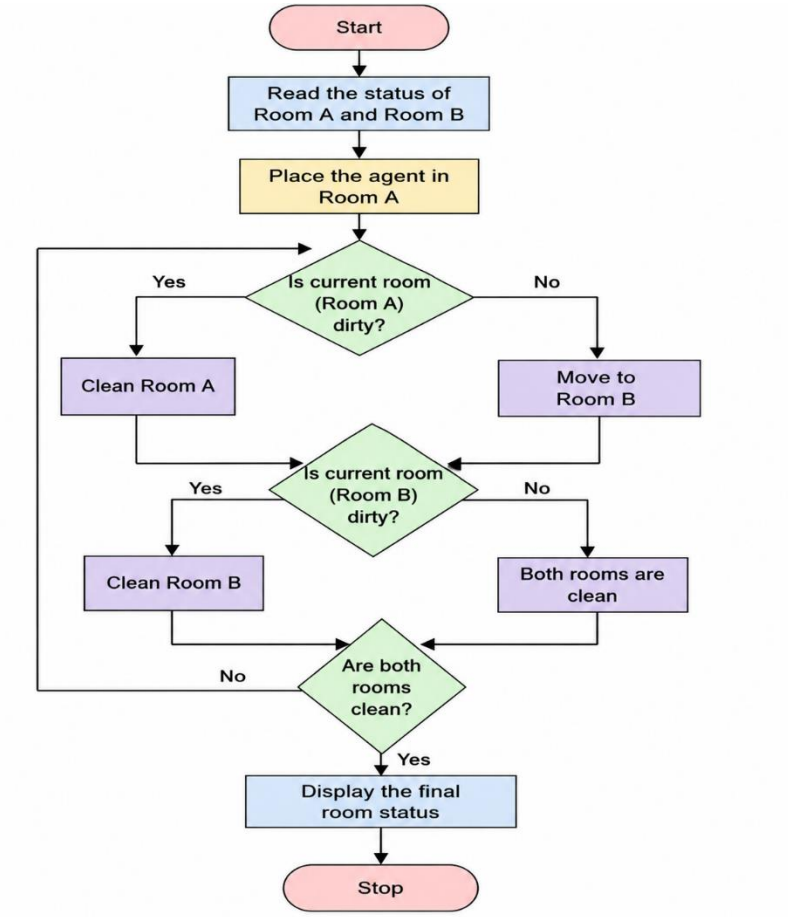
➢ **Rational Behavior**

A **Rational Agent** chooses the best action to achieve its goal.

For the Vacuum Cleaner World:

- Clean dirty rooms.
- Avoid cleaning already clean rooms.
- Stop when all rooms are clean.

Flowchart :



Conclusion : ----------

PRACTICAL NO 2

Aim

To implement a **Model-Based Intelligent Agent** that navigates a grid environment with obstacles using an internal state representation.

Objectives

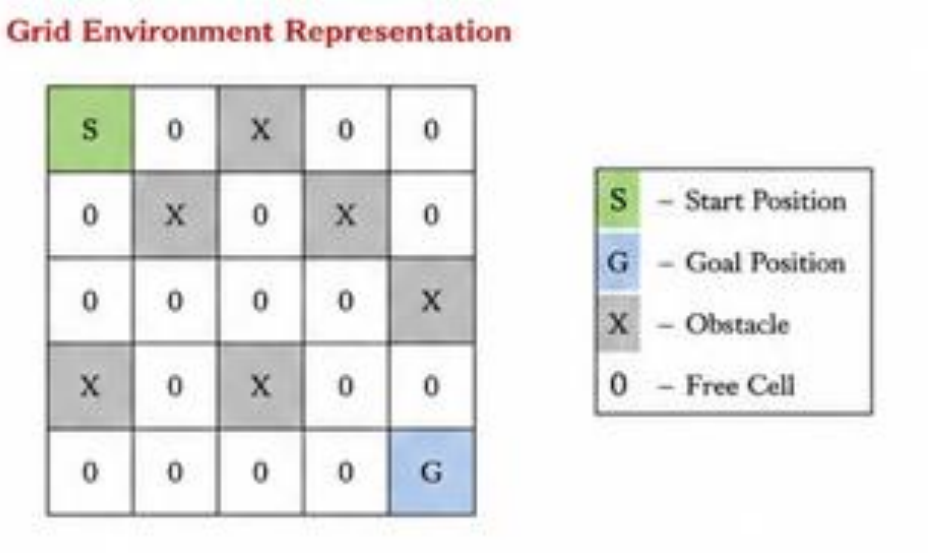
- To understand the concept of a Model-Based Intelligent Agent.
- To represent the environment using a grid.
- To navigate from the start position to the goal while avoiding obstacles.
- To use internal state (memory) for making decisions.

Theory

A **Model-Based Intelligent Agent** is an intelligent agent that maintains an **internal state (memory)** of the environment. Unlike a **Simple Reflex Agent**, it remembers previous information and uses it to make better decisions.

In this practical, the environment is represented as a **grid** consisting of free cells and obstacle cells. The agent starts from a given position and moves towards the goal while avoiding obstacles. The agent keeps track of the cells it has already visited, which forms its internal state. This prevents the agent from revisiting the same cells repeatedly and helps it find a valid path efficiently.

The grid contains:

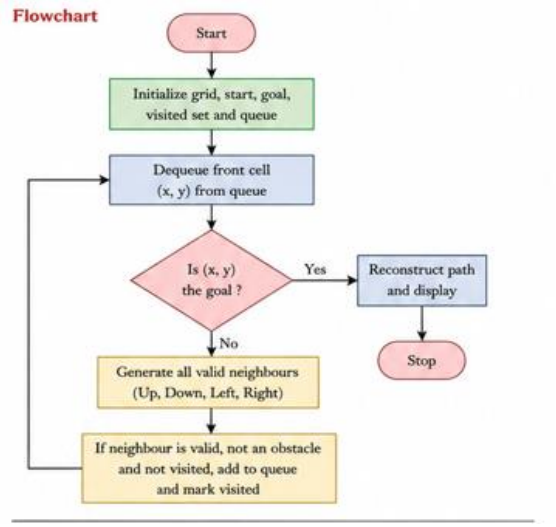


- **S** – Start Position
- **G** – Goal Position
- **X** – Obstacle
- **0** – Free Cell

The agent can move **Up, Down, Left,** or **Right**. It checks whether the next cell is valid before moving. The navigation continues until the goal is reached or no valid path exists.

Algorithm

1. **Start.**
2. **Create the grid environment with obstacles.**
3. **Set the start and goal positions.**
4. **Initialize the internal state (visited set) with start position.**
5. **Initialize a queue for BFS and insert start position.**
6. **While the queue is not empty:**
   a. Dequeue the front cell.
   b. If the cell is the goal, reconstruct and display the path.
   c. Otherwise, generate all valid neighbouring cells (Up, Down, Left, Right).
   d. If a neighbour is inside the grid, not an obstacle and not visited, add it to the queue and mark visited.
7. **If goal is not found, print "No Path Found".**
8. **Stop**



Conclusion

The Model-Based Intelligent Agent was successfully implemented using Python. The agent used its internal state (visited cells) to navigate the grid while avoiding obstacles and successfully reached the goal.

Practical no-03

Aim

To implement the **A* Search Algorithm** for solving the **8-Puzzle Problem** using an appropriate heuristic function.

Objectives

- To understand the concept of the A* Search algorithm.
- To solve the 8-Puzzle problem using heuristic search.
- To find the optimal path from the initial state to the goal state.
- To understand the role of heuristic functions in Artificial Intelligence.

Theory

The **8-Puzzle Problem** is one of the most popular search problems in Artificial Intelligence. It consists of a **3 × 3 grid** containing **eight numbered tiles (1 to 8)** and **one blank space (0)**. The objective is to move the tiles one at a time until the puzzle reaches the desired goal configuration.

The **A* (A-Star) Search Algorithm** is an informed search algorithm that finds the shortest path to the goal state by using a heuristic function. It selects the node with the minimum evaluation function.

The evaluation function is:

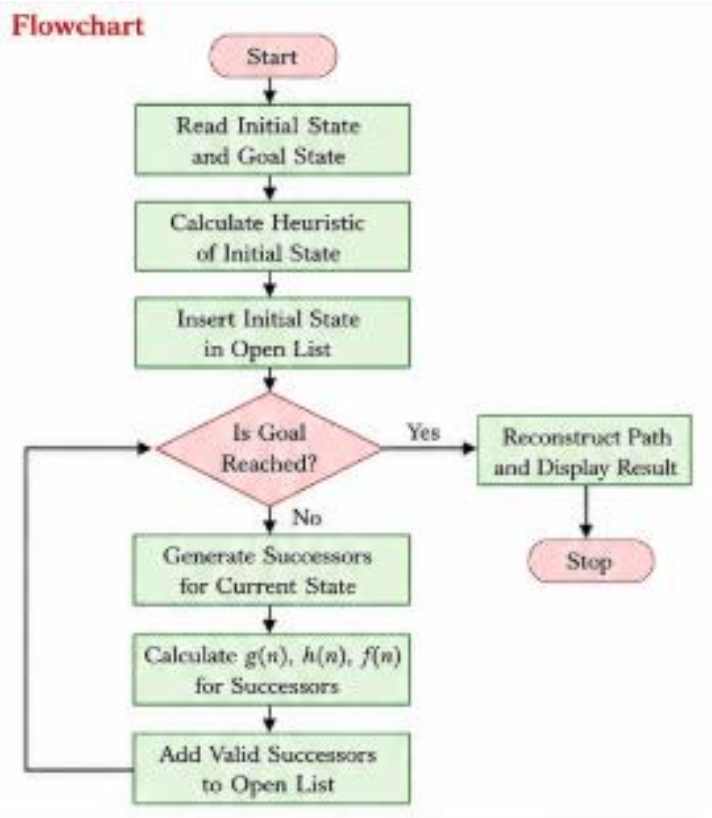
f(n) = g(n) + h(n)

Where:

- **g(n)** = Cost from the start node to the current node.
- **h(n)** = Estimated cost from the current node to the goal (Heuristic).
- **f(n)** = Total estimated cost.

A commonly used heuristic for the 8-Puzzle problem is the **Manhattan Distance**, which calculates the total distance of each tile from its correct position.

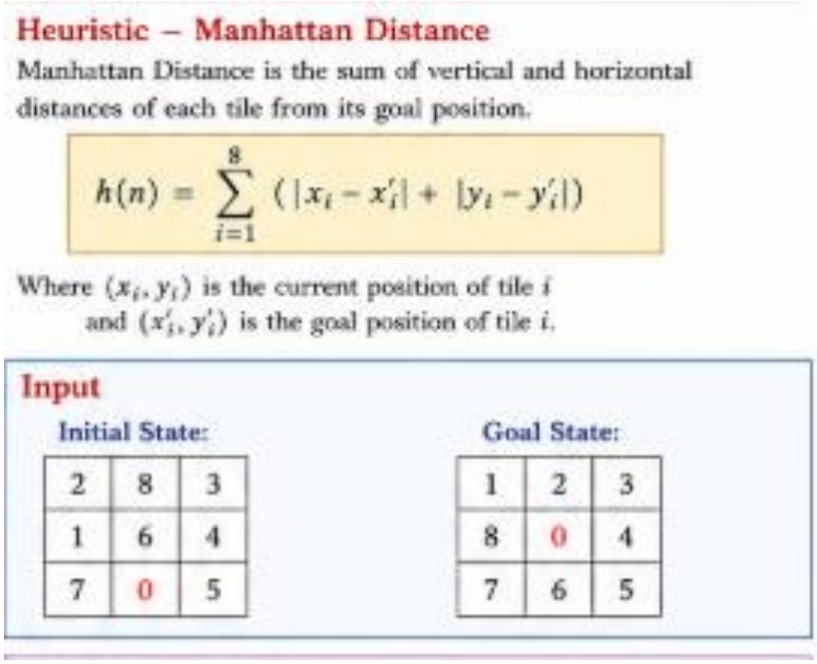
The A* algorithm always expands the node with the smallest value of **f(n)** until the goal state is reached.
The program is implemented in **Python 3** using the **A* Search Algorithm**. The initial and goal states of the puzzle are represented as 3 × 3 matrices. The program calculates the heuristic value using the **Manhattan Distance** method. The algorithm generates all possible moves of the blank tile (up, down, left, and right). It calculates the evaluation function **f(n) = g(n) + h(n)** for each generated state and selects the state with the minimum cost. This process continues until the goal state is reached. Finally, the program displays the sequence of moves required to solve the puzzle.



Algorithm

1. Start.
2. Read the initial state and goal state.
3. Calculate the heuristic value.
4. Store the initial state in the open list.
5. Select the state having the minimum **f(n)** value.
6. Generate all possible successor states.

7. Calculate **g(n), h(n), and f(n)** for each successor.
8. Repeat the process until the goal state is reached.
9. Display the solution path.
10. Stop.



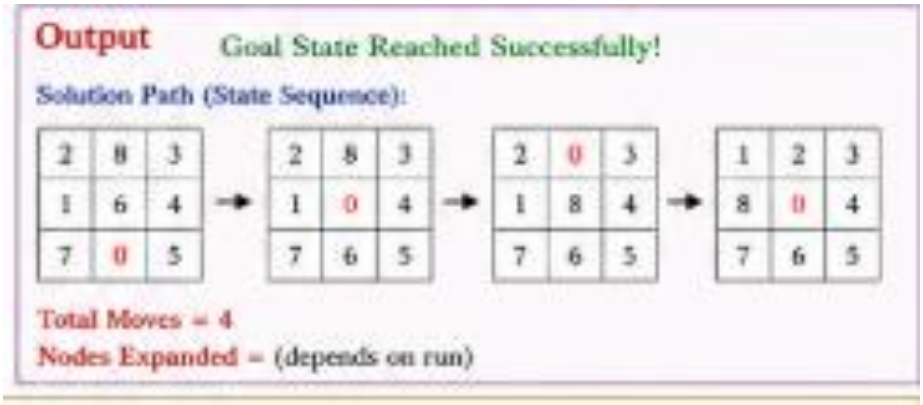
Input

Initial State:

2 8 3
1 6 4
7 0 5

Goal State:

1 2 3
8 0 4
7 6 5



Output

Initial State:
2 8 3
1 6 4
7 0 5

Goal State Reached Successfully!

Solution Found.

Conclusion

The **A* Search Algorithm** was successfully implemented to solve the **8-Puzzle Problem**. The algorithm used a heuristic function to efficiently find the optimal path from the initial state to the goal state.

Practical no -4

Aim

To implement the **Minimax algorithm with Alpha-Beta Pruning** for adversarial search in a Tic-Tac-Toe game.

Objectives

- To understand the concept of adversarial search.
- To study the Minimax algorithm.
- To implement Alpha-Beta Pruning for efficient game tree search.
- To determine the optimal move for a game-playing agent.

Theory

Adversarial Search is used in Artificial Intelligence to solve competitive problems where two players have opposite goals. It is commonly used in games such as Tic-Tac-Toe, Chess, and Checkers.

The **Minimax algorithm** is a decision-making algorithm used in two-player games. It assumes that one player tries to maximize the score (MAX player) while the other tries to minimize it (MIN player). The algorithm explores the game tree and selects the best possible move.

Alpha-Beta Pruning is an optimization technique for the Minimax algorithm. It reduces the number of nodes evaluated by eliminating branches that cannot affect the final decision. This makes the search faster while producing the same optimal result.

The algorithm uses two values:

-
- **Alpha (α):** Best value found for the MAX player.
- **Beta (β):** Best value found for the MIN player.

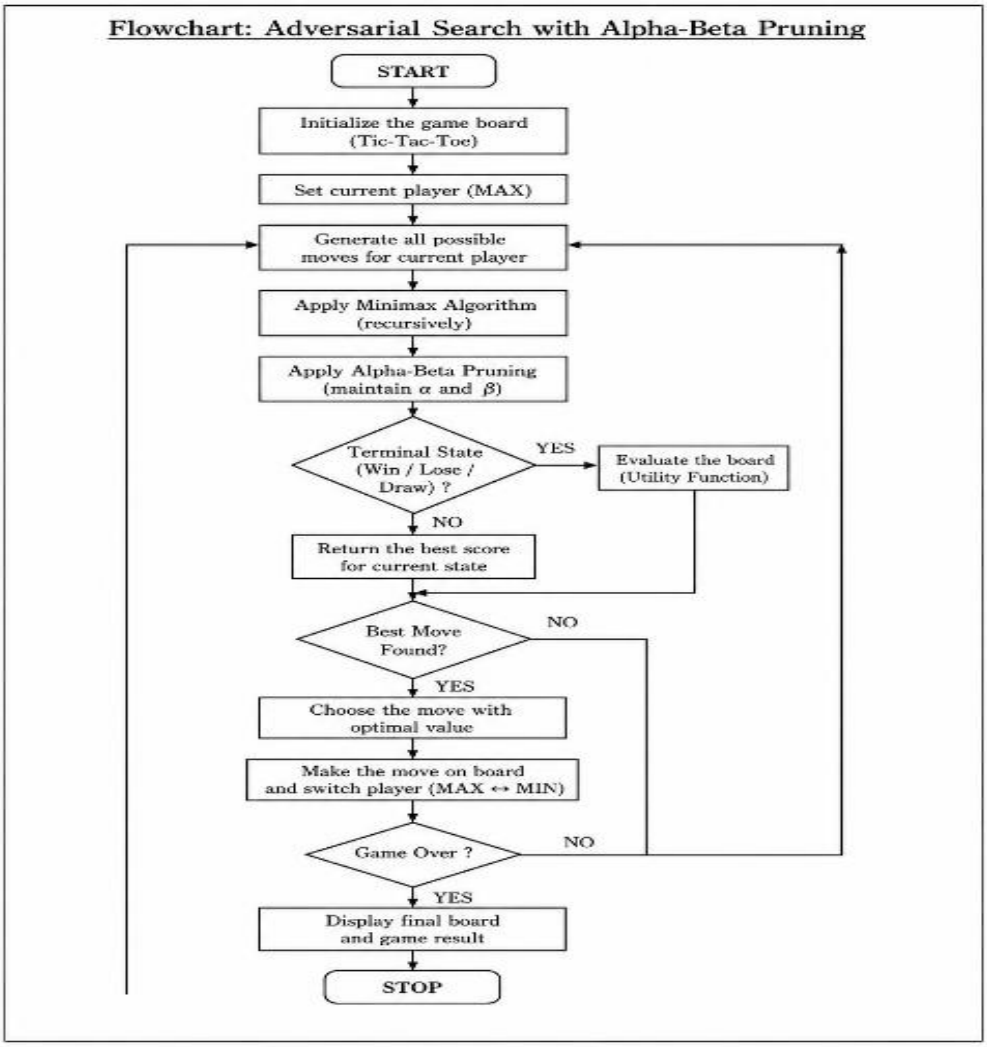
Whenever $\alpha \geq \beta$, the remaining branches are pruned because they cannot improve the result.

The program is implemented in **Python 3** using the **Minimax algorithm** with **Alpha-Beta Pruning**. The game board is represented as a 3 × 3 matrix. The algorithm recursively evaluates all possible moves and updates the alpha and

beta values. Unnecessary branches are pruned to reduce computation. Finally, the AI selects the optimal move that maximizes its chances of winning.

Algorithm

1. Start.
2. Initialize the Tic-Tac-Toe board.
3. Check whether the game has reached a terminal state.
4. Apply the Minimax algorithm.
5. Maintain Alpha and Beta values.
6. Prune branches whenever Alpha ≥ Beta.
7. Select the best move.
8. Display the updated board.
9. Repeat until the game ends.
10. Stop.



Conclu

The Minimax algorithm with Alpha-Beta Pruning was successfully implemented. The AI efficiently selected the optimal move by pruning unnecessary branches, reducing the search time.